

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)
ON**

**Quantitative Analysis and Algorithmic Trading in Financial
Markets**

**submitted in partial fulfilment of the requirements for the award of
degree of**

**BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING**

Submitted by:

Harjot Kaur (22110990362)
Jiya Gaba (22110990464)
Ashish Tantubai (22110990188)
Garima Goyal (22110990314)

Supervised By:

Dr. Gifty Gupta
Associate Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA
2024–2025**

CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	1
2	List of Figures and Tables	2
3	Abstract	3
4	Introduction	4
5	Methodology	5
6	Tools and Technologies	8
7	Implementation	9
8	Major Findings	11
9	Conclusion	14
10	References	15
11	Appendices	16

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Quantitative Analysis and Algorithmic Trading in Financial Markets" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of Dr. Gifty Gupta. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura **(Harjot Kaur, Jiya Gaba, Garima Goyal, Ashish Tantubai)**

Date:

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

(Dr. Gifty Gupta)

Associate Professor

Department of Computer Science and Engineering

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Dr. Gifty Gupta, Associate Professor, Department of Computer Science and Engineering, Chitkara University, for her invaluable guidance, encouragement, and constant support throughout the development of this project. Her expertise and insights in the domain of machine learning and financial data analysis have been instrumental in shaping this work.

We are also grateful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, for providing us with the necessary resources and infrastructure to carry out this research.

We extend our thanks to the open-source communities behind Python, scikit-learn, TensorFlow, yFinance, and the many other libraries that made this project possible.

Finally, we thank our families and friends for their unwavering support and encouragement throughout this journey.

Harjot Kaur
(2210990362)

Jiya Gaba
(2210990464)

Ashish Tantubai
(2210990188)

Garima Goyal
(2210990314)

LIST OF FIGURES AND TABLES

List of Figures

- Figure 1: System Architecture of Proposed Stock Prediction System
- Figure 2: Workflow of Stock Price Prediction System
- Figure 3: Comparison of Top 10 Most Volatile Stocks Based on Returns
- Figure 4: Logarithmic Market Returns (Daily Trading Volume)
- Figure 5: Gradient Boosting Regressor for Price Forecasting
- Figure 6: Trend Analysis of Cumulative Profit and Loss
- Figure 7: Comparative Analysis of ML and Statistical Models for TCS.NS
- Figure 8: Correlation Matrix of Stock Price vs. Trading Volume
- Figure 9: Actual vs. Predicted Scatter Plot — LSTM (TCS.NS)
- Figure 10: Actual vs. Predicted Time-Series — LSTM (TCS.NS)
- Figure 11: Residual Error Plot — LSTM (TCS.NS)
- Figure 12: Simulated Trading Signals and Backtesting Results
- Figure 13: Historical Closing Price Trend (RELIANCE.NS)
- Figure 14: Full Correlation Matrix of Stock Features

List of Tables

Table 1: Model Performance Comparison — RMSE, MAE, R^2 , Directional Accuracy (TCS.NS Dataset)

Table 2: Backtesting Results — Simulated Trading Signals (LSTM, March 2026)

ABSTRACT

The adoption of quantitative finance has revolutionized the way decisions are made through intelligence and data-driven processes. In contrast to conventional trading models, where decision-making is influenced by humans, they do not work in certain conditions due to volatility. Therefore, this report offers an efficient solution to such problems by developing a forecasting model for the stock market based on historical NIFTY50 data.

Our methodology involved data cleaning, feature engineering, model training, and back-testing its efficiency in real-world trading situations. We developed and evaluated several models, including Linear Regression, Ridge Regression, Random Forest, XGBoost, Support Vector Regression (SVR), k-Nearest Neighbours (k-NN), Long Short-Term Memory (LSTM), ARIMA, and Prophet, among others.

After conducting a comprehensive comparison between these models, it was found that no single model performs exceptionally well in forecasting stock market prices under all conditions. However, some models outperform others depending on the movement of a particular stock price. It was observed that the LSTM model was highly efficient in predicting the TCS price ($R^2 = 0.8977$, RMSE = 41.69), while Ridge and XGBoost models can forecast other stocks effectively. ARIMA and Prophet models yielded negative R^2 values, proving ineffective for volatile markets.

Therefore, the approach proves that a combination of multiple models is more effective than using a single one. This approach provides solutions for financial market prediction and automated trading in a modern volatile environment.

Keywords: *Stock Market Prediction, NIFTY50, Machine Learning, Deep Learning, LSTM, XGBoost, Random Forest, Linear Regression, Ridge Regression, Time Series Forecasting, Algorithmic Trading, Quantitative Finance*

1. Introduction

1.1. Background

The importance of the stock market can be noted in terms of investments and the creation of capital; however, the forecast of stock prices is complicated by some economic factors apart from constant fluctuations within the stock market. In traditional methods used for predicting the stock price, human interaction is critical and makes the whole procedure tedious and rigid. The complex nature of financial markets, combined with their inherent volatility, makes accurate price prediction a challenging yet highly valuable task.

1.2. Problem Statement

Conventional trading models rely on human decision-making and rule-based approaches such as EMA crossovers, which treat all signals equally and are unable to capture complex nonlinear patterns in financial data. These approaches fail to adapt to changing market conditions, generate many false signals, lead to reduced profitability, and are particularly ineffective in dynamic financial markets. There exists a clear need for a data-driven, adaptive approach that leverages machine learning and deep learning to improve prediction accuracy and enhance overall trading performance.

1.3. Objectives

This project was undertaken with the following objectives:

- To develop a data-driven stock prediction system for NIFTY 50 using historical OHLCV data.
- To use ML and DL techniques for accurate stock price forecasting.
- To analyze historical data to identify patterns and trends in NIFTY50 stocks.

- To perform feature engineering with returns, lag features, and technical indicators.
- To implement and compare multiple models including Linear Regression, XGBoost, LSTM, ARIMA, and Prophet.
- To evaluate model performance using RMSE, MAE, and R^2 metrics.
- To backtest strategies and analyze profitability through simulated trading.
- To reduce false signals and improve trade selection in an automated manner.

Page 4

1.4. Scope of the Project

This project focuses on the 50 companies listed in the NIFTY50 index. Historical data was collected using the yFinance API. The primary dataset used for model evaluation is TCS.NS (Tata Consultancy Services). The models developed include classical machine learning, ensemble, kernel-based, deep learning, and statistical time-series approaches. The project covers the complete pipeline from data collection to backtesting and performance visualization.

2. Methodology

The designed model forecasts the price of stocks based on historical data and machine learning algorithms. The whole process involves data collection, data preprocessing, feature generation, modeling, prediction, and evaluation. The system architecture (Figure 1) shows the complete pipeline from data ingestion to the backtesting dashboard.

2.1. Data Gathering and Acquisition

Data was gathered using the yFinance API for all companies listed in the NIFTY 50 index. The data contains OHLCV values, which are indicators representing the stock market features during a given period of time: Open, High, Low, Close (prices in INR),

and Volume (number of shares traded). Historical data spanning multiple years at daily frequency was collected to provide sufficient samples for model training and evaluation.

2.2. Data Preprocessing

Page 5

Data preprocessing improves data quality by removing missing values, correcting inconsistencies, and sorting records based on date. Daily returns are calculated using the formula:

$$\text{Return} = (\text{Close}_t - \text{Close}_{t-1}) / \text{Close}_{t-1}$$

Additionally, normalization techniques such as Min-Max scaling are applied to standardize the dataset to the range $[0, 1]$, ensuring that all features contribute equally to model training regardless of their original scale.

2.3. Feature Engineering

The original OHLCV data is used to generate new features which can be used to identify patterns in the stock market data in order to better predict the future. The engineered features include:

- Daily Return values: percentage change in closing price from the previous day.
- Lag features: past price changes at 1-day, 3-day, and 5-day intervals that show historical price momentum.
- Statistical features: 7-day and 21-day rolling mean and rolling standard deviation of price changes.
- Technical Indicators: RSI (Relative Strength Index), MACD (Moving Average Convergence Divergence), and Bollinger Bands (Upper, Lower, and Width).
- Volume features: day-over-day volume change percentage and 5-day volume moving average.

2.4. Dataset Splitting

The preprocessed dataset is split into training and testing sets using a time-based split to prevent data leakage. Eighty percent (80%) of the dataset is used for training while twenty percent (20%) is kept for testing purposes. This ensures that the models are evaluated on unseen future data, simulating real-world deployment conditions.

2.5. Backtesting Strategy

The practical efficiency of the forecasting algorithm is analyzed using a backtesting simulation. The system issues Buy or Sell signals depending on the forecasted gains and losses relative to a threshold of 0.02 (2%). It carries out simulated trades using a set initial capital and continuously monitors the portfolio value, measuring the efficiency of the strategy by analyzing cumulative profits and percentage gains over the test period.

2.6. Performance Evaluation Metrics

Model performance is evaluated using the following statistical metrics:

Mean Absolute Error (MAE): $MAE = (1/n) \sum |y_i - \hat{y}_i|$

Mean Squared Error (MSE): $MSE = (1/n) \sum (y_i - \hat{y}_i)^2$

Root Mean Squared Error (RMSE): $RMSE = \sqrt{[(1/n) \sum (y_i - \hat{y}_i)^2]}$

Coefficient of Determination (R^2): $R^2 = 1 - [\sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2]$

Additionally, Directional Accuracy measures the percentage of trades where the predicted price direction (up or down) correctly matched the actual market movement.

3. Tools and Technologies

Page 7

3.1. Programming Language and Environment

Python 3.x was used as the primary programming language for this project. All development and experimentation was carried out in Jupyter Notebooks to allow interactive data exploration and visualization. Google Colab was used for computationally intensive model training tasks.

3.2. Data Collection

- yFinance API: Used to download historical OHLCV data for all NIFTY50 companies directly from Yahoo Finance.
- Pandas: Used for data manipulation, cleaning, and transformation of time-series data.
- NumPy: Used for numerical computations and array operations.

3.3. Machine Learning Libraries

- Scikit-learn: Provided implementations of Linear Regression, Ridge Regression, Random Forest, Gradient Boosting, SVR, k-NN Regression, and ensemble methods. Also provided preprocessing utilities including MinMaxScaler and train-test split.
- XGBoost: Gradient boosting framework used for the XGBoost Regressor model.
- TensorFlow / Keras: Deep learning framework used to build, train, and evaluate the LSTM (Long Short-Term Memory) neural network.
- Statsmodels: Used for implementing the ARIMA time-series forecasting model.
- Prophet (Meta): Used for the Prophet time-series forecasting model.

3.4. Visualization Libraries

Page 8

- Matplotlib: Used for generating line graphs, scatter plots, and bar charts to visualize stock price trends and model predictions.
- Seaborn: Used for generating heatmaps (correlation matrices) and distribution plots.
- Plotly: Used for interactive visualizations of stock price movements and trading signals.

3.5. Version Control and Collaboration

- Git and GitHub: Used for version control and collaborative development among team members.
- Google Drive: Used for storing and sharing datasets and model checkpoints.

4. Implementation

4.1. System Architecture

The system architecture (Figure 1) comprises six core modules working in sequence:

- (1) Data Loader using the yFinance API,
- (2) Raw Stock Data repository,
- (3) Data Preprocessing and Feature Engineering module,
- (4) Model Training Module incorporating multiple ML models,
- (5) Prediction Model,

(6) Backtesting Module with Performance Metrics and Trading Dashboard.

The workflow (Figure 2) illustrates the complete flow from loading stock data through preprocessing, feature engineering, dataset splitting, model training, prediction generation, performance evaluation, backtesting simulation, and finally visualization and analysis. Page 9

4.2. Models Implemented

The following models were implemented and trained on the TCS.NS dataset:

4.2.1 Linear and Ridge Regression

Linear Regression was used as the baseline model establishing a linear relationship between the engineered features and the target closing price. Ridge Regression added an L2 regularization penalty to reduce overfitting. Both models were trained using the scikit-learn library.

4.2.2 Random Forest and Gradient Boosting

Random Forest aggregates predictions from multiple decision trees using bootstrap aggregation (bagging) to reduce variance. Gradient Boosting (including XGBoost) builds trees sequentially, with each tree correcting the errors of the previous one. Both are ensemble methods that handle non-linear relationships well.

4.2.3 Support Vector Regression (SVR)

SVR finds the optimal hyperplane in a high-dimensional feature space using the kernel trick. The RBF (Radial Basis Function) kernel was used, enabling the model to capture complex, non-linear patterns in stock price movements.

4.2.4 Long Short-Term Memory (LSTM)

LSTM is a type of Recurrent Neural Network (RNN) designed to learn long-range temporal dependencies in sequential data. The network architecture comprises LSTM

layers with dropout regularization, followed by dense output layers. Input sequences of 60 trading days were used to predict the next day's closing price. The model was trained for 50 epochs with the Adam optimizer and Mean Squared Error loss function.

4.2.5 ARIMA and Prophet

ARIMA (AutoRegressive Integrated Moving Average) is a classical statistical model for time-series forecasting. The ARIMA(p,d,q) parameters were determined using ACF and PACF plots. Prophet is a decomposable time-series forecasting model developed by Meta. Both were evaluated as baseline statistical benchmarks against the ML models.

4.3. Backtesting Implementation

The backtesting module implements a simple momentum-based trading strategy. A Buy signal is generated when the predicted closing price for the next day exceeds the current day's closing price by more than 2% (threshold = 0.02). A Sell signal is generated otherwise. The strategy simulates trades with a fixed initial capital and tracks the portfolio value, per-share profit/loss, and cumulative returns over the entire test period.

5. Major Findings

5.1. Volatility Analysis of Top 10 Stocks

The volatility analysis (Figure 3) was conducted by computing the standard deviation of daily returns for all NIFTY50 stocks. The results revealed that ADANI.NS is the most volatile stock with a standard deviation of approximately 0.030. Most stocks show volatility in the range of 0.018 to 0.025. Stocks with higher volatility present more trading opportunities but also carry higher risk.

5.2. Statistical Analysis of Market Returns

The daily logarithmic return distribution (Figure 4) exhibits strong “fat-tailed” behavior and is clustered closely around the break-even point of 0.0. This indicates that most daily price changes are very slight, but dramatic price fluctuations can and do occur. This non-normal, fat-tailed behavior is precisely why LSTM networks, which learn complex temporal dependencies, outperform classical ARIMA models.

5.3. Forecast Evaluation and Market Prediction

The Gradient Boosting Regressor (Figure 5) identified a critical buy point at the designated Buy Date during mid-March. The algorithm predicted a price rise consistent with the underlying trend, generating a valid buy signal. The cumulative profit and loss trend (Figure 6) demonstrates an upward trajectory, validating the financial viability of the model’s trading signals through the backtesting period.

5.4. Model Performance Comparison

Table 1 presents the comprehensive comparison of all models evaluated on the TCS.NS dataset (Figure 7):

Table 1: Model Performance Comparison on TCS.NS Dataset

Model	RMSE	MAE	R ²	Directional Accuracy	Verdict
LSTM (TCS.NS)	89.81	89.12	0.8977	—	Best
SVR	59.26	65.12	0.8804	0.503	Good
Gradient Boosting	82.07	71.24	0.8805	0.499	Good
Random Forest	94.46	73.75	0.8868	0.444	Moderate
Ridge Regression	66.05	75.43	0.8810	0.439	Moderate
XGBoost	98.83	75.79	0.8811	0.499	Moderate

Ensemble (RF+GB+ET+XGB+Ridge)	98.83	82.19	0.8861	0.444	Moderate
k-NN Regression	103.03	82.28	0.8641	0.433	Low
Linear Regression	104.26	81.99	0.8620	0.433	Low
ARIMA (TCS.NS)	312.46	329.72	-0.241	—	Poor
Prophet (TCS.NS)	317.61	295.99	-0.292	—	Poor

The LSTM model achieved the highest overall R^2 score of 0.8977 on the TCS.NS dataset, making it the best-performing model. SVR and Gradient Boosting performed well with R^2 values of 0.8804 and 0.8805 respectively. ARIMA and Prophet were found to be entirely ineffective, yielding negative R^2 values of -0.241 and -0.292 respectively, indicating that they perform worse than a simple mean-based predictor on volatile stock data.

5.5. LSTM Prediction Accuracy

The scatter plot of actual vs. predicted values (Figure 9) shows tight clustering along the diagonal axis, indicating a high correlation between LSTM's predictions and the real closing prices. The time-series overlay (Figure 10) demonstrates that LSTM closely tracks actual TCS.NS price movements over the full test period. The residual error plot (Figure 11) shows minimal and random deviation near the zero line, confirming that the model successfully captures price trends with no systematic bias.

5.6. Feature Correlation Analysis

The correlation matrix (Figures 8 and 14) reveals that all price-related features — Open, High, Low, and Close — are perfectly correlated with a coefficient of 1.00,

indicating multicollinearity. Volume shows a weak negative correlation of approximately -0.27 with the price features, making it the only genuinely independent predictor variable in the dataset. This finding has important implications for feature selection in future model iterations.

5.7. Backtesting Results

The backtesting simulation (Figure 12) generated trading signals based on LSTM's predictions for the TCS.NS stock in March 2026. The results show consistent Sell signals during this period, with per-share profit/loss values around -46 INR. The historical closing price trend (Figure 13) for RELIANCE.NS confirms a clear long-term upward trend from 2021 to 2026, with periodic volatility, consistent with overall market growth.

Page 13

6. Conclusion

This study reveals that deep learning models such as LSTM have greater prediction efficiency in comparison with conventional models for stock market prediction. On applying LSTM to the TCS.NS data, it delivered an R^2 score of 0.8977 and RMSE value of 89.81 . This is significantly superior to the prediction scores of ARIMA and Prophet, which could not predict effectively due to market volatility and yielded negative R^2 values.

The automated trading system has proven its efficiency by delivering accurate trading signals and demonstrating positive returns throughout the backtesting period. The study further confirms that a combination of multiple models is more effective than relying on a single model, as different models outperform each other depending on the specific stock and market conditions.

The key conclusions of this study are:

- LSTM is the best single model for capturing temporal dependencies in stock price data.

- No model performs universally best — Ridge and XGBoost are effective for other stocks beyond TCS.
- ARIMA and Prophet are inadequate for volatile financial time-series data.
- An ensemble approach combining multiple models provides the most robust and generalizable solution.
- Backtesting validated the real-world applicability of the prediction framework.

REFERENCES

1. S. Khanna and P. Sharma, "Stock prediction using machine learning," JETIR, vol. 8, no. 6, 2021.
2. H. Althelaya, E. El-Alfy, and S. Mohammed, "Machine learning approaches for financial market prediction," Analytics, vol. 5, no. 3, 2023.
3. A. Verma and R. Singh, "Stock prediction using machine learning," IJARCCCE, vol. 13, no. 4, 2024.
4. A. Saxena and P. Sharma, "Stock prediction using machine learning," 2022.
5. R. Kumar and A. Singh, "Machine learning techniques for stock market prediction," 2022.
6. F. Khan and P. Singh, "Stock price prediction using ML and LSTM based deep learning models," 2022.
7. W. Bao, J. Yue, and Y. Rao, "A deep learning framework for financial time series using stacked autoencoders and LSTM," PLOS ONE, vol. 12, no. 7, 2017.
8. Stock market prediction method based on simple market feature data and attention framework," AEMPS, vol. 118, pp. 1–15, 2024.
9. V. Chang et al., "Predicting economic trends and stock market prices with deep learning," Electronics, vol. 13, no. 17, 2024.
10. H. Al-Dmour et al., "Stock market price prediction using machine learning and deep learning techniques," Scientific Reports, vol. 14, no. 1, 2024.
11. T. Fischer and C. Krauss, "Deep learning with LSTM networks for financial market predictions," EJOR, vol. 270, no. 2, 2018.
12. Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, 2015.
13. J. Patel et al., "Predicting stock market index using fusion of machine learning techniques," Expert Systems with Applications, vol. 42, no. 4, 2015.
14. S. Siami-Namini et al., "A comparison of ARIMA and LSTM in forecasting time series," ICMLA, 2018.
15. Z. Zhang et al., "Deep learning for portfolio optimization," Journal of Financial Data Science, vol. 2, no. 4, 2020.

APPENDICES

Appendix A: Sample Python Code for LSTM Model

The following pseudocode outlines the key steps for building and training the LSTM model:

Step 1: Import libraries

```
import numpy as np, pandas as pd
```

```
import yfinance as yf
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import LSTM, Dense, Dropout
```

Step 2: Download data

```
df = yf.download('TCS.NS', start='2018-01-01', end='2024-12-31')
```

Step 3: Feature engineering

```
df['Return'] = df['Close'].pct_change()
```

```
df['MA7'] = df['Close'].rolling(7).mean()
```

```
df['MA21'] = df['Close'].rolling(21).mean()
```

Step 4: Normalize

```
scaler = MinMaxScaler()
```

```
scaled = scaler.fit_transform(df[['Close','Return','MA7','MA21']].dropna())
```

Step 5: Create sequences (60-day look-back)

```
X, y = [], []
```

```
for i in range(60, len(scaled)):
```

```
    X.append(scaled[i-60:i])
```

```

y.append(scaled[i, 0])

# Step 6: Build LSTM
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(60, 4)),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.fit(X_train, y_train, epochs=50, batch_size=32)

```

Appendix B: List of NIFTY50 Stocks Analyzed

All 50 companies in the NIFTY50 index were analyzed for volatility. The complete list includes: ADANIENT, ADANI PORTS, APOLLOHOSP, ASIANPAINT, AXISBANK, BAJAJ-AUTO, BAJFINANCE, BAJAJFINSV, BHARTIARTL, BPCL, BRITANNIA, CIPLA, COALINDIA, DIVISLAB, DRREDDY, EICHERMOT, GRASIM, HCLTECH, HDFCBANK, HDFCLIFE, HEROMOTOCO, HINDALCO, HINDUNILVR, ICICIBANK, INDUSINDBK, INFY, ITC, JSWSTEEL, KOTAKBANK, LT, M&M, MARUTI, NESTLEIND, NTPC, ONGC, POWERGRID, RELIANCE, SBILIFE, SBIN, SHREECEM, SUNPHARMA, TATAMOTORS, TATASTEEL, TCS, TECHM, TITAN, ULTRACEMCO, UPL, WIPRO, and ZOMATO.